

---

# Práctica 13: reflejos, cuadriláteros y texturas

---

**Fecha de entrega:** 30 de abril de 2026, 13:50

**Objetivo:** Extensión de un trazador de rayos con reflejos

## 1. Instrucciones

En esta práctica partiremos del resultado de la práctica anterior en la que se construyó un trazador de rayos con soporte para luces y sombras e iremos aplicando extensiones encaminadas a añadir soporte para reflejos, cuadriláteros y texturas.

Las extensiones se van describiendo poco a poco más abajo. Aunque se pasará por distintas “versiones” del trazador, sólo será necesario incluir la versión final del código. La entrega debe incluir esos ficheros de tal forma que baste compilar todos los `.cpp` desde el directorio raíz de código (en Linux debería bastar con algo como `g++ *.cpp`).

En la entrega será necesario, además, incluir el resultado de renderizado tras la implementación de algunos de los apartados. En todos los casos la captura será en resolución FullHD (1920×1080) y será una imagen en formato PNG.

## 2. Punto de partida

El punto de partida es la escena final de la práctica anterior:

- La cámara colocada en  $(0, 0, 3)$  mirando hacia  $(0, 0, 0)$  y con una apertura de foco de 60 grados.
- Tres esferas de radio 1, colocadas en  $(-2, 0, -2)$ ,  $(0, 0, -2)$  y  $(2, 0, -2)$ . Los colores de cada una serán roja, amarilla y azul, respectivamente.
- Una esfera verde haciendo las veces de “suelo”, colocada en  $(0, -100, -2)$  y de radio 99.
- Una luz puntual blanca en la posición  $(2, 2, 0)$ .

### 3. Reflejo simple

Añade la posibilidad de que un material refleje parcialmente la luz. Para eso, incorpora un factor de reflexión en el material de forma que cuando se calcula el color del punto, si éste es mayor que 0 se calcule el rayo reflejado en el punto y se obtenga su color. Ese color será entonces multiplicado por el factor y sumado al color final.

```
Color Renderer::shade(Ray r, ShapeIntersection hit) {

    Color ret(BLACK);

    // Ambiente
    ret += ...; // Luz de ambiente, cableada

    // luces
    para cada luz l en el mundo:
        ...

    // reflejo
    si hit.material.factorReflejo > 0:
        Ray secundario = ray(hit.pos, reflect(hit.pos, hit.normal));
        ret += hit.material.factorReflejo * ray_color(secundario);

    return ret;
}
```

Prueba la nueva característica incorporando al suelo (esfera verde) un factor de reflejo de 0.5.

Genera una imagen con el resultado de nombre `01_ReflejoSimple.png` e incorpórala a la entrega.

### 4. Reflejo múltiple

Cambia la escena para que los materiales del resto de esferas tengan también un factor de reflejo de 0.5.

Para evitar una posible recursión infinita, limita el número de llamadas a, por ejemplo, 10.

Genera una imagen con el resultado (`02_Reflejos.png`) e incorpórala a la entrega.

### 5. Cuadriláteros

Extiende el trazador para soportar, además de esferas, cuadriláteros.

Para eso, crea una nueva clase, hermana de la clase de la esfera, que reciba en el constructor un punto en 3D con la esquina del cuadrilátero y dos vectores (no paralelos) que definan los lados adyacentes a la esquina. Debe, además, recibir el material del que está formado.

Para implementar la intersección entre un rayo y un cuadrilátero te puedes basar en la sección 6 del libro *Ray Tracing: The Next Week*<sup>1</sup>.

---

<sup>1</sup>En caso de usarlo, ten cuidado con la fórmula de  $w$  del final de la sección 6.5 en donde simplifica  $(u \times v)$  por  $n$ , pues eso solo se cumple si  $u$  y  $v$  son unitarios.

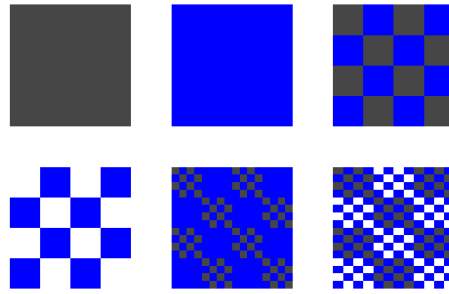


Figura 1: Distintas texturas

Para probarlo, puedes sustituir la esfera verde que hace las veces de suelo por un cuadrilátero con una esquina en  $(3, -1, 5)$  lados  $(-6, 0, 0)$  y  $(0, 0, 6)$ .

## 6. Texturas

Incorpora al trazador soporte básico para texturas. El objetivo no es tener texturas cargadas desde ficheros de imágenes (aunque puedes hacerlo para el último apartado) sino poder tener materiales cuyo color no sea constante.

Incorpora una clase abstracta para representar una textura que tenga un método:

```
virtual Color color(float u, float v) const = 0;
```

Implementa una clase hija que defina una textura de color constante (indicado en el constructor) y otra que construya una textura tipo cuadrícula: en el constructor define la textura utilizada en cada uno de los dos tipos de cuadros y el número de filas y columnas que tiene.

A modo de ejemplo, el código siguiente genera las texturas que pueden verse en la figura 1.

```
ConstantTexture gris(Color(0.25, 0.25, 0.25));
ConstantTexture azul(Color(0, 0, 1));
CheckerTexture patron1(gris, azul, 4, 4);
CheckerTexture patron2(ConstantTexture(1,1,1), azul, 4, 4);
CheckerTexture tex1(patron1, azul, 4, 4);
CheckerTexture tex2(patron1, patron2, 3, 4);
```

Extiende la clase `ShapeIntersection` para que en el cálculo de la intersección entre rayo y geometría se devuelva también las coordenadas de textura (para la esfera revisa la sección 4.4 del libro *Ray Tracing: The Next Week*). Por último, extiende el material de la práctica anterior para que la función que devuelve el color reciba un objeto de `ShapeIntersection` y para que en su construcción pueda configurarse con una textura.

Aplica una textura con un patrón al suelo de la escena del apartado anterior genera una imagen con el resultado (`03_Final.png`) e incorpórala a la entrega.

## 7. Aplica tu imaginación

Aprovecha las nuevas características y crea alguna escena nueva y renderízala. Incorpóralas a la entrega en un directorio `EscenasAdicionales` incluyendo tanto el PNG con el resultado final como el `main.cpp` que construye la escena.

## 8. Entrega de la práctica

La práctica debe entregarse utilizando el mecanismo de entrega del campus virtual, no más tarde de la fecha y hora indicada en la cabecera de la práctica.

Solo uno de los dos miembros del grupo debe hacer la entrega, subiendo al campus un fichero llamado `GrupoNN.zip` donde NN representa el número de grupo con dos dígitos.

El fichero debe tener exclusivamente:

- Un directorio `Src` con el código fuente completo del trazador de rayos. El `main` debe construir una escena similar a la del apartado 8. El código debe compilar tal y como se ha descrito en la sección 1.
- Las imágenes indicadas en los apartados anteriores.
- Un fichero `alumnos.txt` donde se indicará el nombre de los componentes del grupo.